

Command sequence in `main_interfaz.f90`

This document describes the sequence of operations performed by `main_interfaz` (source file `main_interfaz.f90`). When the executable is run, the user is guided through the steps below via a series of prompts in the terminal. Inputs are read from standard input through the helper subroutine `read_clean_line`, which skips blank lines and full-line comments (lines whose first non-blank character is "!" or "#"), so the input file can be annotated and reused via input redirection:

```
> .\interfaz_remix.exe < datos_interfaz_denit_2reacts.txt (run from cmd.exe)
```

All error/EOF conditions are handled gracefully and any non-positive time step is rejected.

Pre-process

1. Query IEEE floating-point support. If available, switch to abrupt underflow (`ieee_set_underflow_mode` with `gradual = .false.`) and clear the underflow, inexact, overflow, divide-by-zero and invalid exception flags so that benign exceptions do not abort the run.
2. Print introductory message: *"Esta es la interfaz para resolver la parte reactiva de una iteración de mezcla reactiva en 1D usando Euler explícito."*
3. Create chemistry class object `my_chem`.

Input reading

4. Read directory of chemical databases (`dir_DB`), including a trailing backslash or forward slash depending on the OS.
5. Read directory of the problem to be solved (`dir_pb`), with the corresponding trailing slash.
6. Read the common root of the input and output file names (`root_files`).
7. Read the number of targets in the mesh (`num_tar`) in response to the prompt *"Cuantos targets tiene la malla?"*.
8. Read chemistry: `my_chem` calls `read_chemistry_interface`, passing the trimmed root, problem directory, database directory and `num_tar`. The latter is used to validate the target waters file against the mesh size.

9. Read the name of the file where `my_chem` will write the aqueous component concentrations of the initial and external water types (`file_u_wat_types`).
10. Get the number of aqueous components in the first target water: `my_chem` calls `get_num_aq_comps_tar_wat`. The value is stored in `num_aq_comps` and is used later to choose the interface and to read the `u_tilde` input file.
11. Write the initial and external water-type concentrations: `my_chem` calls `write_conc_comp_tar_wat`. The program prints a confirmation message of the form *"Archivo <file_u_wat_types> generado correctamente."*
12. Read the name of the file where `my_chem` will write the concentrations after each reactive mixing iteration (`file_u_new`).
13. Read the name of the file containing the aqueous component concentrations after solving an iteration of conservative transport (`file_u_tilde`). The file must live in the problem directory. Rows correspond to aqueous components and columns to targets.
14. Read the input-matrix layout flag (`flag_transpose`) indicating whether the `file_u_tilde` file has rows as targets and columns as components (`flag_transpose = 1`) or the canonical layout with rows as components and columns as targets (`flag_transpose = 0`). Any value other than 0 or 1 aborts execution with the error *"Opcion no valida. Tiene que ser 1 (matriz de componentes transpuesta) o 0 (no transpuesta)."*
15. Read the initial time step (`Delta_t`). Execution is aborted with the error *"El paso de tiempo debe ser mayor que cero"* if the value is not strictly positive.
16. Read whether the time step is constant (`flag_Delta_t = 1`) or variable (`flag_Delta_t = 0`). Any value other than 0 or 1 aborts execution with the error *"Opcion no valida. Tiene que ser 1 (paso de tiempo constante) o 0 (variable)."*

Process

17. Trim the `u_tilde` input filename and select the target of the procedure pointer `p_interfaz` depending on the chemistry:
 - `interfaz_esp_arch` — if the first target water has no equilibrium reactions.
 - `interfaz_comps_arch_eq / interfaz_comps_arch_eq_T` — if there are equilibrium reactions but no kinetics (`_T` variant used when `flag_transpose = 1`).
 - `interfaz_comps_arch_eq_kin / interfaz_comps_arch_eq_kin_T` — if there are both equilibrium and kinetic reactions (`_T` variant used when `flag_transpose = 1`).

18. Print a reminder that the file with the conservative transport concentrations must be updated by the user before each iteration.
19. Reactive mixing loop:
 - Compute one reactive mixing iteration: call `p_interfaz` with `my_chem`, `dir_pb`, `num_aq_comps`, `file_u_tilde`, `Delta_t` and `file_u_new`.
 - Ask the user whether to continue iterating (`flag = 1`) or stop (`flag = 0`). Any other value aborts execution with the error *"Opción no válida. Tiene que ser 1 o 0."*
 - If the time step is variable (`flag_Delta_t = 0`) and the user chose to continue, read the new value of `Delta_t`; the new value must also be strictly positive.

Robust I/O and input comments

20. Every interactive read goes through the internal subroutine `read_clean_line(buf, ios)`, which reads the next line from standard input into a 512-character buffer, skipping blank lines and lines whose first non-blank character is `"!"` or `"#"`. The returned buffer is then parsed by an internal list-directed read of the form `read(buf, *, iostat=ios) <variable>`, which preserves the original type-safe conversion to integers, reals (e.g. `1d-2`) and character strings.
21. Only full-line comments are stripped: `"!"` or `"#"` appearing in the middle of a line are kept verbatim, so file paths containing those characters are preserved. Comments must therefore be placed on their own line.
22. Every interactive read is guarded by an `iostat` check on both the line-read and the internal read. On read failure or end-of-file, the program prints a message suggesting to run in an interactive terminal (or redirect input from `fort.5`) and calls the internal `safe_stop(1)` subroutine, which clears pending IEEE flags before stopping with exit code 1.

Post-process

23. If IEEE support is available, clear the floating-point exception flags (via the internal `clear_ieee_flags` subroutine) before exiting.
24. Print termination message *"El programa ha terminado."* and end the program.